Encadenamiento hacia adelante:

02_forward_chaining.py --query "AccesoNoche(Ana)"

```
(venv) PS C:\Users\HP240G7\documents\Ejercicio_9\Project> python 02_forward_chaining.py --query "AccesoNoche(Ana)"
Forward chaining
------------------------------------------------
Agenda (given facts): Estudiante(Ana), Estudiante(Luis), Profesor(Marta), TieneCredencial(Ana), TieneCredencial(Marta),
HorarioNocturno(Hoy)
Query: AccesoNoche(Ana)
--- round 1 ---
Fire R_miembro_est  {x_1/Ana}  ⊢  Miembro(Ana)
Fire R_miembro_prof  {x_2/Marta}  ⊢  Miembro(Marta)
Fire R_beca  {x_4/Ana}  ⊢  Becario(Ana)
--- round 2 ---
Fire R_acceso  {x_7/Ana}  ⊢  AccesoNoche(Ana)
Fire R_acceso  {x_7/Marta}  ⊢  AccesoNoche(Marta)
--- round 3 ---
No new facts.
Query proved: AccesoNoche(Ana)
------------------------------------------------
Proved: YES
Inferred facts: Miembro(Ana), Miembro(Marta), Becario(Ana), AccesoNoche(Ana), AccesoNoche(Marta)
```

02_forward_chaining.py --query "AccesoNoche(Marta)"

```
(venv) PS C:\Users\HP240G7\documents\Ejercicio_9\Project> python 02_forward_chaining.py --query "AccesoNoche(Marta)"
Forward chaining
------------------------------------------------
Agenda (given facts): Estudiante(Ana), Estudiante(Luis), Profesor(Marta), TieneCredencial(Ana), TieneCredencial(Marta),
HorarioNocturno(Hoy)
Query: AccesoNoche(Marta)
--- round 1 ---
Fire R_miembro_est  {x_1/Ana}  ⊢  Miembro(Ana)
Fire R_miembro_prof  {x_2/Marta}  ⊢  Miembro(Marta)
Fire R_beca  {x_4/Ana}  ⊢  Becario(Ana)
--- round 2 ---
Fire R_acceso  {x_7/Ana}  ⊢  AccesoNoche(Ana)
Fire R_acceso  {x_7/Marta}  ⊢  AccesoNoche(Marta)
--- round 3 ---
No new facts.
Query proved: AccesoNoche(Marta)
------------------------------------------------
Proved: YES
Inferred facts: Miembro(Ana), Miembro(Marta), Becario(Ana), AccesoNoche(Ana), AccesoNoche(Marta)
```

02_forward_chaining.py --query "AccesoNoche(Luis)"

```
(venv) PS C:\Users\HP240G7\documents\Ejercicio_9\Project> python 02_forward_chaining.py --query "AccesoNoche(Luis)"
Forward chaining
------------------------------------------------
Agenda (given facts): Estudiante(Ana), Estudiante(Luis), Profesor(Marta), TieneCredencial(Ana), TieneCredencial(Marta),
HorarioNocturno(Hoy)
Query: AccesoNoche(Luis)
--- round 1 ---
Fire R_miembro_est  {x_1/Ana}  ⊢  Miembro(Ana)
Fire R_miembro_prof  {x_2/Marta}  ⊢  Miembro(Marta)
Fire R_beca  {x_4/Ana}  ⊢  Becario(Ana)
--- round 2 ---
Fire R_acceso  {x_7/Ana}  ⊢  AccesoNoche(Ana)
Fire R_acceso  {x_7/Marta}  ⊢  AccesoNoche(Marta)
--- round 3 ---
No new facts.
Query not proved.
------------------------------------------------
Proved: NO
Inferred facts: Miembro(Ana), Miembro(Marta), Becario(Ana), AccesoNoche(Ana), AccesoNoche(Marta)
```

Encadenamiento hacia atras:

03_backward_chaining.py --query "AccesoNoche(Ana)"

```
(venv) PS C:\Users\HP240G7\documents\Ejercicio_9\Project> python 03_backward_chaining.py --query "AccesoNoche(Ana)"
Backward chaining
------------------------------------------------
Query: AccesoNoche(Ana)
goal  AccesoNoche(Ana)
rule   R_acceso  {x_3/Ana}
  goal  Miembro(Ana)
  rule   R_miembro_est  {x_3/Ana, x_4/Ana}
    goal  Estudiante(Ana)
    fact   Estudiante(Ana)  {x_3/Ana, x_4/Ana}
    goal  TieneCredencial(Ana)
    fact   TieneCredencial(Ana)  {x_3/Ana, x_4/Ana}
  goal  HorarioNocturno(Hoy)
  fact   HorarioNocturno(Hoy)  {x_3/Ana, x_4/Ana}
Query proved.
------------------------------------------------
Proved: YES
Goals visited: AccesoNoche(Ana), Miembro(Ana), Estudiante(Ana), TieneCredencial(Ana), HorarioNocturno(Hoy)
```

03_backward_chaining.py --query "AccesoNoche(Marta)"

```
(venv) PS C:\Users\HP240G7\documents\Ejercicio_9\Project> python 03_backward_chaining.py --query "AccesoNoche(Marta)"
Backward chaining
------------------------------------------------
Query: AccesoNoche(Marta)
goal  AccesoNoche(Marta)
rule   R_acceso  {x_3/Marta}
  goal  Miembro(Marta)
  rule   R_miembro_est  {x_3/Marta, x_4/Marta}
    goal  Estudiante(Marta)
    fail   Estudiante(Marta)
  rule   R_miembro_prof  {x_3/Marta, x_9/Marta}
    goal  Profesor(Marta)
    fact   Profesor(Marta)  {x_3/Marta, x_9/Marta}
  goal  HorarioNocturno(Hoy)
  fact   HorarioNocturno(Hoy)  {x_3/Marta, x_9/Marta}
Query proved.
------------------------------------------------
Proved: YES
Goals visited: AccesoNoche(Marta), Miembro(Marta), Estudiante(Marta), Profesor(Marta), HorarioNocturno(Hoy)
```

03_backward_chaining.py --query "AccesoNoche(Luis)"

```
(venv) PS C:\Users\HP240G7\documents\Ejercicio_9\Project> python 03_backward_chaining.py --query "AccesoNoche(Luis)"
Backward chaining
------------------------------------------------
Query: AccesoNoche(Luis)
goal  AccesoNoche(Luis)
rule   R_acceso  {x_3/Luis}
  goal  Miembro(Luis)
  rule   R_miembro_est  {x_3/Luis, x_4/Luis}
    goal  Estudiante(Luis)
    fact   Estudiante(Luis)  {x_3/Luis, x_4/Luis}
    goal  TieneCredencial(Luis)
    fail   TieneCredencial(Luis)
  rule   R_miembro_prof  {x_3/Luis, x_9/Luis}
    goal  Profesor(Luis)
    fail   Profesor(Luis)
  fail   Miembro(Luis)
fail   AccesoNoche(Luis)
Query not proved.
------------------------------------------------
Proved: NO
Goals visited: AccesoNoche(Luis), Miembro(Luis), Estudiante(Luis), TieneCredencial(Luis), Profesor(Luis)
```

REPORTE:

En encadenamiento hacia adelante parte desde los hechos y va probando las reglas para poder inferir si se puede dar acceso a la persona, en cambio Encadenamiento hacia atrás parte desde Acceso Noche y va buscando las reglas que coincidan, como para tener acceso te piden ser miembro, pues busca quienes son miembros en este caso los miembros son Estudiantes que tienen credencial, como el Encadenamiento hacia atrás no visita todas las reglas como el de adelante y como ninguna regla para acceder de noche te pide beca, pues en Encadenamiento para atrás no saldrá que Ana es becaria. Igualmente por lo mismo que la búsqueda para atrás no parte de los hechos si no que del query , este no tiene idea que Marta es profesora y por eso lo prueba, en estas pruebas tenemos que Luis no tiene credencial por lo tanto se le niega el acceso, pero esto cambiaría si en los hechos agregamos que tenga credencial, como abajo muestro, en la corrida de Forward chaining aparecen como nuevos átomos “Miembro(Luis)”, “Becari(Luis)” y “Acceso Noche(Luis)”, ya que al ser un hecho que tiene credencial, ya puede inferir que es un miembro, al ser miembro puede tener acceso y aparte también al ser estudiante y tener credencial puede tener beca.

python 04_compare.py --kb kb/biblioteca.yaml

```
Forward chaining
--------------------------------------------------
Agenda (given facts): Estudiante(Ana), Estudiante(Luis), Profesor(Marta), TieneCredencial(Ana), TieneCredencial(Marta),
HorarioNocturno(Hoy), TieneCredencial(Luis)
Query: AccesoNoche(Luis)
--- round 1 ---
Fire R_miembro_est  {x_1/Ana}  ⊢  Miembro(Ana)
Fire R_miembro_est  {x_1/Luis}  ⊢  Miembro(Luis)
Fire R_miembro_prof  {x_2/Marta}  ⊢  Miembro(Marta)
Fire R_beca  {x_4/Ana}  ⊢  Becario(Ana)
Fire R_beca  {x_4/Luis}  ⊢  Becario(Luis)
--- round 2 ---
Fire R_acceso  {x_7/Ana}  ⊢  AccesoNoche(Ana)
Fire R_acceso  {x_7/Luis}  ⊢  AccesoNoche(Luis)
Fire R_acceso  {x_7/Marta}  ⊢  AccesoNoche(Marta)
--- round 3 ---
No new facts.
Query proved: AccesoNoche(Luis)

Backward chaining
--------------------------------------------------
Query: AccesoNoche(Luis)
goal  AccesoNoche(Luis)
rule   R_acceso  {x_3/Luis}
  goal  Miembro(Luis)
  rule   R_miembro_est  {x_3/Luis, x_4/Luis}
    goal  Estudiante(Luis)
    fact   Estudiante(Luis)  {x_3/Luis, x_4/Luis}
    goal  TieneCredencial(Luis)
    fact   TieneCredencial(Luis)  {x_3/Luis, x_4/Luis}
  goal  HorarioNocturno(Hoy)
  fact   HorarioNocturno(Hoy)  {x_3/Luis, x_4/Luis}
Query proved.

Contrast
--------------------------------------------------
Query AccesoNoche(Luis):  FC=YES  BC=YES
```